Computer Vision Assignment 1

Submission Requirements

Format

1. Project materials including template: `proj1.zip`.
2. Please use the provided `proj1_report.pptx` template to write your report (anwser all questions), and convert the slide deck into a PDF for your final submission. The report should contain your name, student ID, and email address.
3. You should provide a README file to describe how to run the code.
4. Pack your `proj1_report.pdf`, `proj1_code` and `README.md` into a zip file, named with your student ID, like `[student ID].zip`.
5. **Note that, additional Python packages should not be introduced. Do not use absolute paths in your code. Use relative paths like the starter code already does.**

Submission Way

1. Please submit to https://box.nju.edu.cn/u/d/15430c4f3aa44c5da66d/

2. The deadline is 23:59 on April 30, 2026. No submission after this deadline is acceptable.

About Plagiarize

DO NOT PLAGIARIZE! We have no tolerance for plagiarizing and will penalize it with giving zero score. You may refer to some others' materials, please make citations such that one can tell which part is actually yours.

Evaluation Criterion

We mainly evaluate your submission according to your code and report. Efficient implementation, elegant code style, concise and logical report are all important factors towards a high score.

Setup

1. Install Anaconda.
2. Download and extract the project starter code.
3. Create a conda environment using the appropriate command. On Windows, open the installed "Conda prompt" to run the command. On MacOS and Linux, you can just use a terminal window to run the command. Modify the command based on your OS (linux, mac, or win): `conda env create -f proj1_env_<OS>.yml`
4. This will create an environment named "cv_proj1". Activate it using the Windows command, `activate cv_proj1` or the MacOS / Linux command, `conda activate cv_proj1` or `source activate cv_proj1`.

5. Install the project package, by running `pip install -e .` inside the repo folder. This might be unnecessary for every project, but is good practice when setting up a new `conda` environment that may have `pip`requirements.
6. Run the notebook using `jupyter notebook ./proj1_code/proj1.ipynb`.
7. After implementing all functions, ensure that all sanity checks are passing by running `pytest proj1_unit_tests` inside the repo folder.
8. Generate the zip folder for the code portion of your submission once you've finished the project.

Part 1

1.1 Gaussian kernels

Gaussian filters are used for blurring images. You will first implement create `Gaussian_kernel_1D()`, a function that creates a 1D Gaussian vector according to two parameters: the kernel size (length of the 1D vector) and σ, the standard deviation of the Gaussian. The vector should have values populated from evaluating the 1D Gaussian pdf at each coordinate. The 1D Gaussian is defined as:

$$p(x; \mu, \sigma^2) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{1}{2\sigma^2}(x-\mu)^2\right)$$

Next, you will implement create `Gaussian_kernel_2D()`, which creates a 2-dimensional Gaussian kernel according to a free parameter, cutoff frequency, which controls how much low frequency to leave in the image. Choosing an appropriate cutoff frequency value is an important step for later in the project when you create hybrid images. We recommend that you implement create `Gaussian_kernel_2D()` by creating a 2D Gaussian kernel as the outer product of two 1D Gaussians, which you have now already implemented in create `Gaussian_kernel_1D()`. This is possible because the 2D Gaussian filter is separable. The multivariate Gaussian function is defined as:

$$p(x;\mu,\Sigma) = \frac{1}{(2\pi)^{n/2}\det(\Sigma)^{1/2}} \exp\left(-\frac{1}{2}(x-\mu)^\top \Sigma^{-1}(x-\mu)\right)$$

where n is equal to the dimension of x, μ is the mean coordinate (where the Gaussian has peak value), and $\sum$ is the covariance matrix.

You will use the value of cutoff frequency to define the size, mean, and variance of the Gaussian kernel. Specifically, the kernel G should be size $(k;\ k)$ where $k = 4 \times \text{cutoff frequency} + 1$, have peak value at $\mu = \lfloor 2k \rfloor$, standard deviation $\sigma = \text{cutoff frequency}$, and values that sum to 1, i.e., $\sum_{ij} G_{ij} = 1$. If your kernel doesn't sum to 1, you can normalize it as a postprocess by dividing each value by the sum of the kernel.

1.2 Image Filtering

Image filtering (or convolution) is a fundamental image processing tool. See chapter 3.2 of Szeliski and the lecture materials to learn about image filtering (specifically linear filtering). You will be writing your own function to implement image filtering from scratch. More specifically, you will implement `my_conv2d_numpy()` which imitates the `filter2D()` function in the OpenCV library. As specified in `part1.py`, your filtering algorithm must: (1) support grayscale and color images, (2) support arbitrarily-shaped filters, as long as both dimensions are odd (e.g., 7 × 9 filters, but not 4 × 5 filters), (3) pad the input image with zeros, and (4) return a filtered image which is the same resolution as the input image. We have provided an iPython notebook, `proj1.ipynb` and some unit tests (which are called in the notebook) to help you debug your image filtering algorithm. Note that there is a time limit of 5 minutes for a single call to `my_conv2d_numpy()`, so try to optimize your implementation if it goes over.

After achieving the aforementioned objectives, you may notice that the filtering operation introduces black artifacts (because the filter window falls off the edge of the image). Therefore, you need to implement an optimized version to address this issue. Please conduct your own research and analysis, and document your solution

and discussion in the report.

1.3 Hybrid Images

A hybrid image is the sum of a low-pass filtered version of one image and a high-pass filtered version of another image. As mentioned above, *cutoff frequency* controls how much high frequency to leave in one image and how much low frequency to leave in the other image. In `cutoff_frequencies.txt`, we provide a default value of 7 for each pair of images (the value of line i corresponds to the cutoff frequency value for the i-th image pair). You should replace these values with the ones you find work best for each image pair. In the paper it is suggested to use two cutoff frequencies (one tuned for each image), and you are free to try that as well. In the starter code, the cutoff frequency is controlled by changing the standard deviation of the Gaussian filter used in constructing the hybrid images. You will first implement `create_hybrid_image()` according to the starter code in `part1.py`. Your function will call `my_conv2d_numpy()` using the kernel generated from `create_Gaussian_kernel()` to create low and high frequency images, and then combine them into a hybrid image.

1.4 More Hybrid Image Pairs

Apart from the image pairs already provided in the project files (dog & cat, etc.), there are many other images that can be combined to create hybrid images.

In this section, you are required to find at least one such pair of images from daily life or the internet to create your own custom hybrid image. In your report, please analyze what factors you believe contribute to the differences in the results produced by various image pairs.

Part 2

2.1 Dataloader

You will now implement creating hybrid images again but using PyTorch. The `HybridImageDataset` class in `part2_datasets.py` will create tuples using pairs of images with a corresponding cutoff frequency (which you should have found from experimenting in Part 1). The image paths will be loaded from `data/` using `make_dataset()` and the cutoff frequencies from `cutoff_frequencies.txt` using get cutoff frequencies(). Additionally, you will implement `__len__()`, which returns the number of image pairs, and `__getitem__()`, which returns the i-th tuple. Refer to this tutorial for additional information on data loading and processing.

2.2 Model

Next, you will implement the HybridImageModel class in `part2_models.py`. Instead of using your implementation of `my_conv2d_numpy()` to get the low and high frequencies from a pair of images, `low_pass()` should use the 2D convolution operator from `torch.nn.functional` to apply a low pass filter to a given image. You will have to implement `get_kernel()` which calls your `create_Gaussian_kernel()` function from `part1.py` for each pair of images using the cutoff frequencies as specified in `cutoff_frequencies.txt`, and reshape it to the appropriate dimensions for PyTorch. Then, similar to `create_hybrid_image()` from `part1.py`, `forward()` will call `get_kernel()` and `low_pass()` to create the low and high frequency images, and combine them into a hybrid image. Refer to this tutorial for additional information on defining neural networks using PyTorch.

You will compare the runtimes of your hybrid image implementations from Parts 1 and 2.

Part 3

You will now implement `my_conv2d_pytorch()` in `part3.py` using the same 2D convolution operator from `torch.nn.functional` used in `low_pass()`.

Before we proceed, here are two quick definitions of terms we'll use often when describing convolution:

1. **Stride**: When the stride is 1 then we move the filters one pixel at a time. When the stride is 2 (or uncommonly 3 or more, though this is rare in practice) then the filters jump 2 pixels at a time as we slide them around.
2. **Padding**: The amount of pixels added to an image when it is being convolved with a the kernel. Padding can help prevent an image from shrinking during the convolution operation.

Unlike `my_conv2d_numpy()` from `part1.py`, the shape of your output does not necessarily have to be the same as the input image. Instead, given an input image of shape (1, d_1, h_1, w_1) and kernel of shape (N, $\frac{d_1}{g}$, k, k), your output will be of shape (1, d_2, h_2, w_2) where g is the number of groups, $d_2 = N$, $h_2 = \frac{h_1 - k + 2 \times p}{s} + 1$, and $w_2 = \frac{w_1 - k + 2 \times p}{s} + 1$ and p and s are padding and stride, respectively.

Think about why the equations for output width w_2 and output height h_2 are true - try sketching out a 5×5 grid, and seeing how many places you can place a 3×3 square within the grid with stride 1. What about with stride 2? Does your finding match what the equation states?

We demonstrate the effect of the value of the `groups` parameter on a simple example with an input image of shape (1, 2, 3, 3) and a kernel of shape (4, 1, 3, 3):

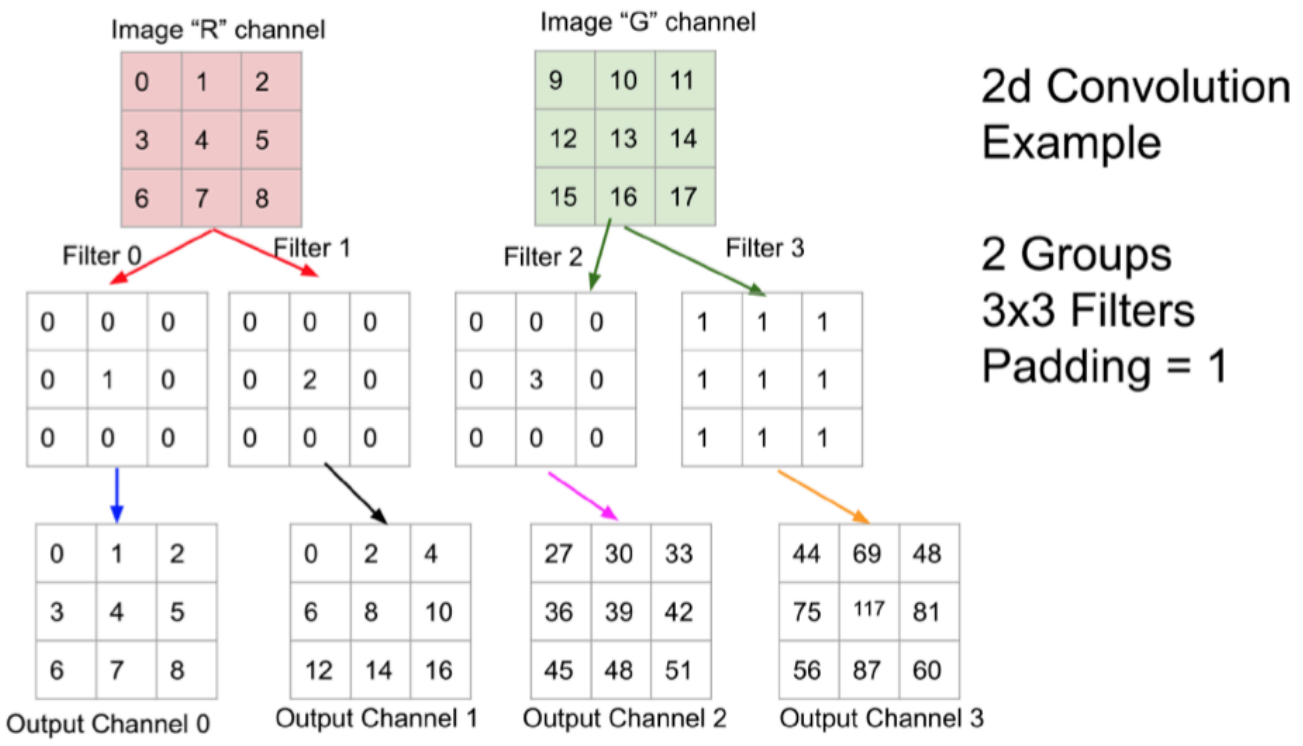

Figure 1: Visualization of a simple example using groups=2.

Data

We provide you with 5 pairs of aligned images which can be merged reasonably well into hybrid images. The alignment is super important because it affects the perceptual grouping (read the paper for details). We encourage you to create additional examples (e.g., change of expression, morph between different objects, change over time, etc.).

For the example shown below:

Figure 2: Look at the image from very close, then from far away.

The two original images look like this:

(a) Dog (b) Cat

Figure 3:

The low-pass (blurred) and high-pass version of these images look like this (Figure 4):

(a) Low frequencies of dog image.

(b) High frequencies of cat image.

Figure 4:

The high frequency image in Figure 4b is actually zero-mean with negative values, so it is visualized by adding 0.5. In the resulting visualization, bright values are positive and dark values are negative.

Adding the high and low frequencies together (Figures 4b and 4a, respectively) gives you the image in Figure 2. If you're having trouble seeing the multiple interpretations of the image, a useful way to visualize the effect is by progressively downsampling the hybrid image, as done in Figure 5. The starter code provides a function, `vis_image_scales_numpy()` in `utils.py`, which can be used to save and display such visualizations.

Figure 5:

Potential usefull NumPy functions

`np.pad()`, which does many kinds of image padding for you, `np.clip()`, which "clips" out any values in an array outside of a specified range, `np.sum()` and `np.multiply()`, which makes it efficient to do the convolution (dot product) between the filter and windows of the image. Documentation for NumPy can be found here or by Googling the function in question.

Forbidden functions

(You can use these for testing, but not in your final code). Anything that takes care of the filter operation or creates a 2D Gaussian kernel directly for you is forbidden. If it feels like you're sidestepping the work, then it's probably not allowed.

Part 4: Frequency Compression and Image Filtering

1. Problem Statement

In this part, you will explore frequency-based compression using Fourier transforms and low-pass filtering. You are required to:

1. Implement frequency-based compression with different retention ratios (retaining different proportions of low frequencies).
2. Evaluate the quality of reconstructed images using PSNR (Peak Signal-to-Noise Ratio).

You can use `numpy.fft`, such as `numpy.fft.fft2`, `numpy.fft.ifft2`, and `numpy.fft.fftshift`. Additionally, common `numpy` operations include `np.log10`, `np.clip`, `np.abs`, `np.mean`, `np.square`, and `np.sqrt`, etc.

2. Background

Frequency Compression:
Frequency compression involves reducing the amount of high-frequency information in an image, which corresponds to fine details and noise. By retaining only low-frequency components, we can achieve data compression while preserving the overall structure of the image.
Low-Pass Filtering:
low-pass filter allows only low frequencies to pass through while attenuating or removing high frequencies.
Evaluation Metric: PSNR
PSNR (Peak Signal-to-Noise Ratio) is a widely used metric to measure the quality of the reconstructed image compared to the original image. It quantifies the difference between the original and reconstructed images in terms of pixel intensity. A higher PSNR value indicates better reconstruction quality. However, PSNR does not always correlate perfectly with human perception of image quality, as it only considers pixel-wise differences.

$$\mathrm{PSNR} = 10 \cdot \log_{10} \left(\frac{\mathrm{MAX}^2}{\mathrm{MSE}} \right) \tag{1}$$

where:

- MAX = Maximum pixel value (e.g., 255 for 8-bit images, 65535 for 16-bit images).
- MSE = Mean Squared Error between the original and reconstructed images.

3. Task Description

Task 1: Frequency-Based Compression

1. Implement frequency compression using **2D Fourier Transform** and **low-pass filtering**.
2. Retain different proportions of low frequencies (retention ratios): `[0.1, 0.3, 0.5, 0.7]`.
3. Reconstruct the compressed images using the inverse Fourier transform.
4. Calculate and report the PSNR for each retention ratio.

Task 2 (Discussion): How to Reduce Retention Ratio while Maintaining Quality?
This is a discussion question. Think critically about strategies to reduce the **retention ratio** (i.e., retain fewer frequency components) while maintaining similar visual quality or achieving similar PSNR (Peak Signal-to-Noise Ratio). The goal is to provide ideas, not full implementations.